Why These Losses?

A probabilistic refresher for Deep Learning

Prof. Nipun Batra

ES 667 · Deep Learning · IIT Gandhinagar

Three familiar equations

You have already met these losses:

$$\mathcal{L}_{\text{reg}} = \frac{1}{n}\sum_i (y_i - \hat{y}_i)^2 \qquad \mathcal{L}_{\text{class}} = -\frac{1}{n}\sum_i \log p_\theta(y_i \mid x_i)$$

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda\|\theta\|_2^2$$

Why exactly these three expressions? Today: they are not arbitrary.

Were these arbitrary choices?

Q

Which is it?

- Historical convention?
- Convenient gradients?
- Empirically successful?
- **Derived from probability?**
- All of the above?

Reveal. Primarily **consequences of probabilistic assumptions** — that is the thread for the whole lecture.

The map for today

V

Assumption about outputs

$$p_\theta(y \mid x)$$

likelihood ↓ negative log-likelihood

training loss

Assumption about parameters

$$p(\theta)$$

Bayes' rule ↓ MAP

regularization

Minimal probability revision

A model should describe uncertainty

$$\hat{y} = f_\theta(x)$$

a single number

$$Y \mid X\{=\}x \sim p_\theta(\cdot \mid x)$$

a full distribution

- regression network → a **Gaussian mean**
- classifier → **categorical** probabilities
- language model → categorical distribution over **tokens**

A neural network parameterizes a probability distribution.

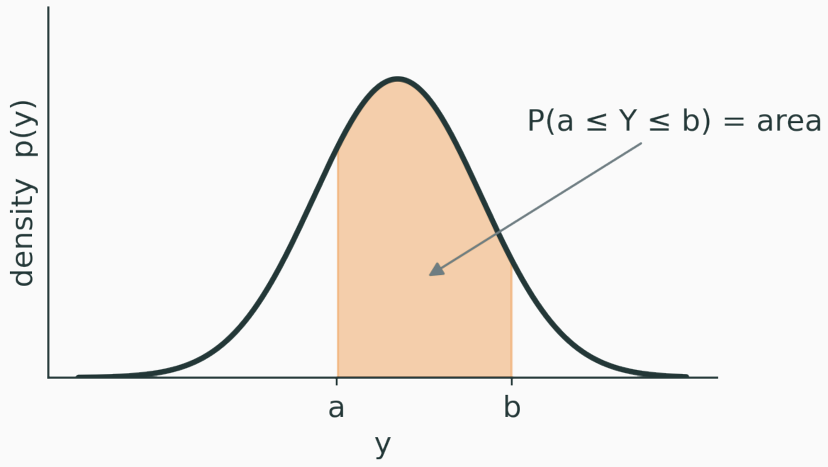
Density is not probability

V

For a continuous Y, probability comes from **area**:

$$P(a \leq Y \leq b) = \int_a^b p(y)\,\mathrm{d}y$$

$p(y) \neq P(Y\{=\}y)$ — a density can **exceed 1**; only *integrals* are probabilities.



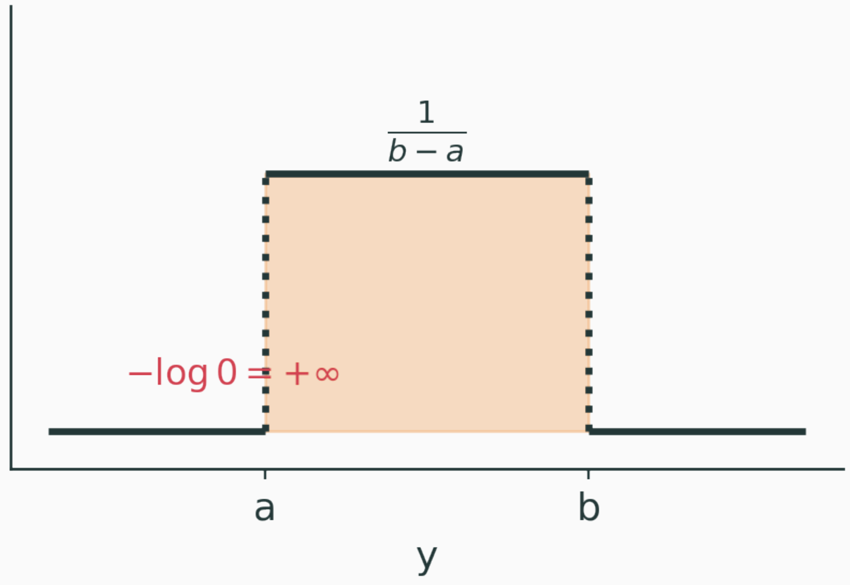
Uniform distribution: support matters

V

$$Y \sim \mathcal{U}(a,b) \quad p(y) = \begin{cases} 1/(b-a) a \le y \le b \\ 0 \quad \text{otherwise} \end{cases}$$

Q. NLL *outside* $[a,b]$? **A.** $-\log 0 = +\infty$.

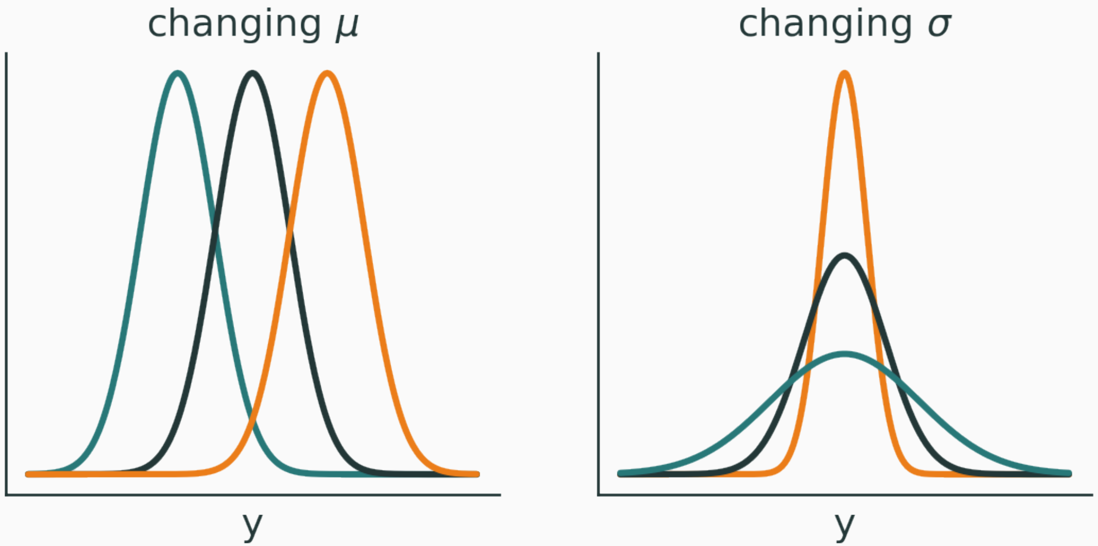
A likelihood can impose a **hard constraint**.



Gaussian distribution

V

$$Y \sim \mathcal{N}(\mu, \sigma^2) \qquad p(y) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y-\mu)^2}{2\sigma^2}\right)$$



Only two knobs: **location** μ and **scale** σ.

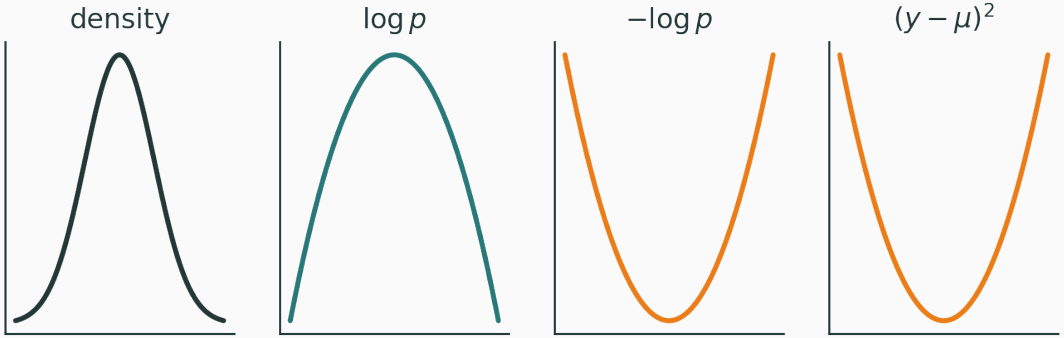
Why the Gaussian produces a square

D

Take the negative log:

$$-\log p(y) = \underbrace{\frac{(y-\mu)^2}{2\sigma^2}}_{\text{depends on } \mu} + \underbrace{\frac{1}{2}\log(2\pi\sigma^2)}_{\text{constant in } \mu}$$

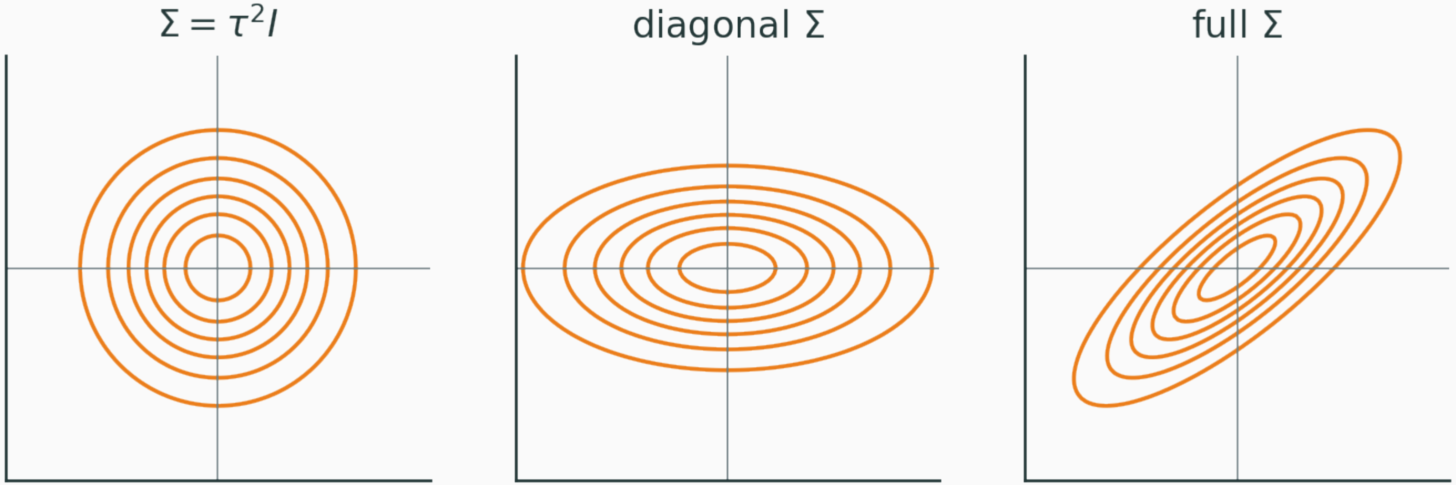
The only μ-dependent term is the **squared error** $(y-\mu)^2$.



Gaussian → log → negative log → squared error

Multivariate Gaussian geometry

$$\theta \sim \mathcal{N}(\mu, \Sigma) \quad -\log p(\theta) = \frac{1}{2}(\theta - \mu)^\top \Sigma^{-1}(\theta - \mu) + C$$



Covariance says **which directions in parameter space are plausible.**

Independence turns products into sums

Conditionally independent observations:

$$p(\mathcal{D} \mid \theta) = \prod_{i=1}^{n} p_\theta(y_i \mid x_i)$$

Take logs — the product becomes a **sum**:

$$\log p(\mathcal{D} \mid \theta) = \sum_{i=1}^{n} \log p_\theta(y_i \mid x_i)$$

Why a loss is an **average over examples**, and why minibatches work: $\frac{1}{|B|} \sum_{i \in B} \ell_i$.

Maximum likelihood → training losses

Likelihood: one expression, two viewpoints

Q

The one model $p_\theta(y \mid x)$, read two ways:

Fix θ, vary y
→ a **distribution** over outcomes.

Fix observed y, vary θ
→ a **likelihood** over parameters.

The likelihood is *not* “the probability that θ is correct.”

Maximum likelihood estimation

$$\hat{\theta}_{\mathrm{MLE}} = \arg\max_{\theta} p(\mathcal{D} \mid \theta)$$

Interpretation. Choose the parameters under which the observed dataset would be **least surprising**.

From MLE to negative log-likelihood

D

$$\arg\max_\theta \prod_i p_\theta(y_i \mid x_i) = \arg\max_\theta \sum_i \log p_\theta(y_i \mid x_i)$$

$$= \arg\min_\theta \left(-\sum_i \log p_\theta(y_i \mid x_i) \right)$$

$$\boxed{\ell_i(\theta) = -\log p_\theta(y_i \mid x_i)}$$

NLL is empirical risk

ML notation — empirical risk:

$$\hat{R}(\theta) = \frac{1}{n} \sum_i \ell(y_i, f_\theta(x_i))$$

Probabilistic reading of the per-example loss:

$$\ell(y_i, f_\theta(x_i)) = -\log p_\theta(y_i \mid x_i)$$

Choosing a loss = choosing a probabilistic observation model.

Regression — where does MSE come from?

Regression as signal plus noise

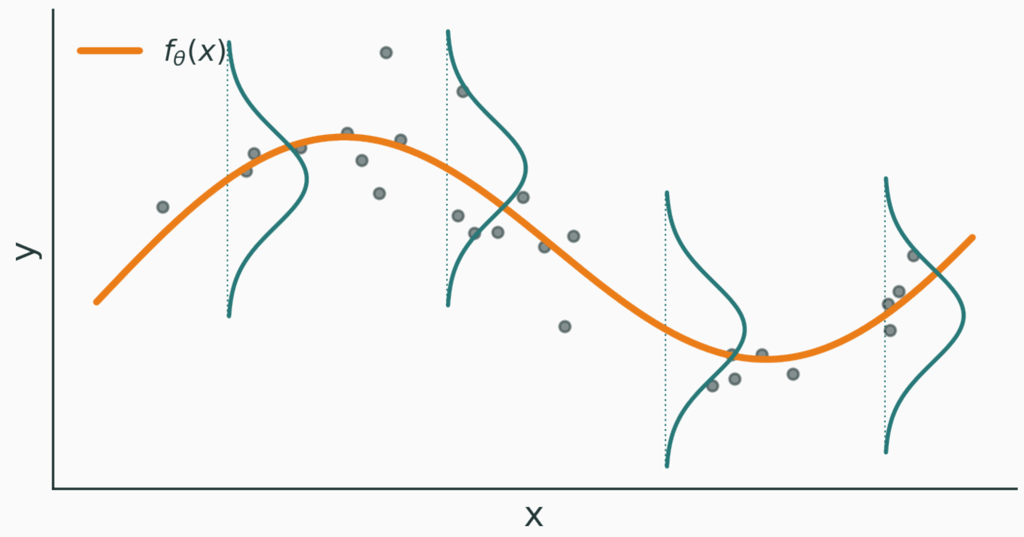
$$y_i = f_\theta(x_i) + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma^2)$$

Therefore the conditional output distribution is

$$Y_i \mid x_i, \theta \sim \mathcal{N}\big(f_\theta(x_i), \sigma^2\big)$$

What the assumption means visually

V



Gaussian noise lives in the **output direction**, conditional on x — a vertical bell at every input.

D

$$p_\theta(y_i \mid x_i) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - f_\theta(x_i))^2}{2\sigma^2}\right)$$

Negative log of a single example:

$$-\log p_\theta(y_i \mid x_i) = \frac{(y_i - f_\theta(x_i))^2}{2\sigma^2} + C$$

MSE is Gaussian MLE

Sum over the dataset:

$$-\log p(\mathcal{D} \mid \theta) = \frac{1}{2\sigma^2} \sum_i (y_i - f_\theta(x_i))^2 + C$$

With σ fixed, the minimizer is

$$\hat{\theta}_{\text{MLE}} = \arg\min_\theta \sum_i (y_i - f_\theta(x_i))^2$$

MSE = Gaussian NLL, up to constants.

What role does σ^2 play?

optional

$$\ell_i = \frac{(y_i - \mu_i)^2}{2\sigma^2} + \frac{1}{2}\log\sigma^2 + C$$

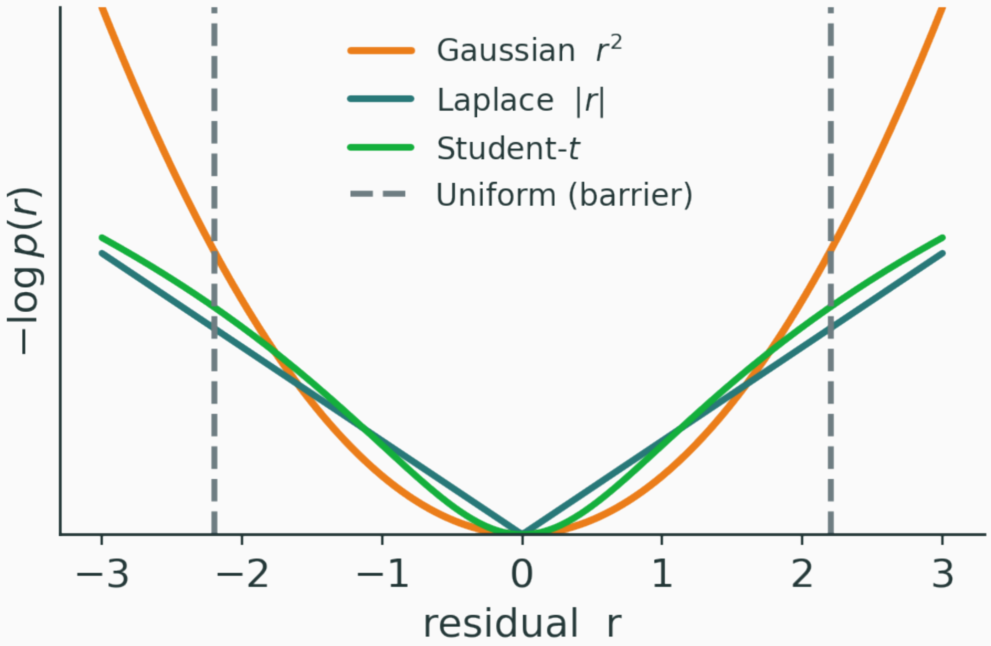
- small σ: errors penalized **strongly**
- large σ: errors deemed **less surprising**
- a predicted σ_i needs the $\frac{1}{2}\log\sigma_i^2$ term — it stops the model claiming infinite uncertainty

Heteroscedastic regression: let the network output both $(\mu_i, \log\sigma_i^2) = f_\theta(x_i)$.

V

Noise model	**NLL**
Gaussian	r^2
Laplace	$\|r\|$
Uniform	hard interval
Student-t	robust

$r = y - \hat{y}$



I

INTERACTIVE

`likelihood-to-loss.html` — three synchronized panels: the **noise density** $p(r)$, the **loss** $-\log p(r)$, and **data with a fitted line**.
Controls: Gaussian / Laplace / Uniform / Student-t · noise scale · slope & intercept · **add an outlier** · per-point contributions.

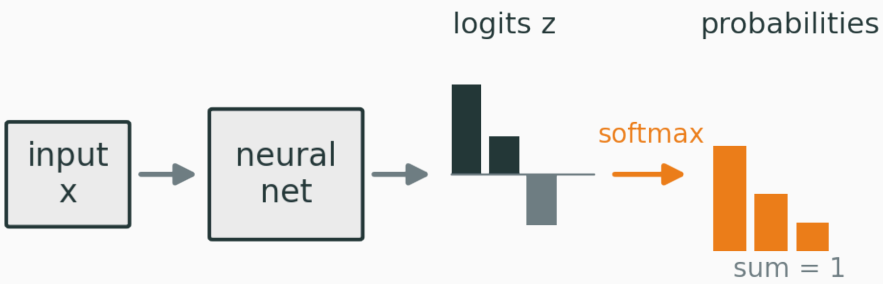
Ask: which model reacts most to an outlier? why does Uniform give an infinite barrier? which loss is robust?

Classification and cross-entropy

Classification should output probabilities

For K classes:

$$p_\theta(y\{=\}k \mid x) \geq 0, \qquad \sum_{k=1}^{K} p_\theta(y\{=\}k \mid x) = 1$$



logits $z_k \in \mathbb{R} \rightarrow$ **probabilities** $p_k \in [0, 1]$.

Binary classification: Bernoulli model

$$Y \mid x \sim \mathrm{Bernoulli}(p_\theta(x))$$

Compact one-line form:

$$p(y \mid x, \theta) = p^y (1-p)^{1-y}$$

$y\{=\}1 \Rightarrow p(y)\{=\}p$

$y\{=\}0 \Rightarrow p(y)\{=\}1 - p$

Bernoulli NLL gives BCE

D

$$-\log p(y \mid x, \theta) = -\log\left(p^y (1-p)^{1-y}\right)$$

$$= -y \log p - (1-y)\log(1-p)$$

Binary cross-entropy = Bernoulli NLL.

Why use logits?

V

The network emits an unbounded score $z = f_\theta(x) \in \mathbb{R}$; the **sigmoid** maps it to a probability:

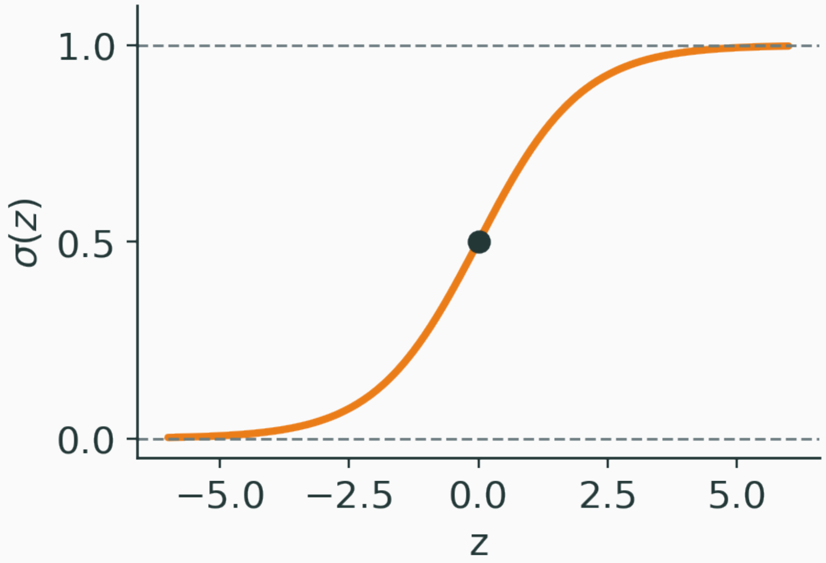
$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$z \to -\infty : p \to 0$

$z = 0 : p = 0.5$

$z \to +\infty : p \to 1$

In practice sigmoid + BCE are fused for numerical stability.



Multi-class: categorical model

$$Y \mid x \sim \mathrm{Categorical}(p_1, ..., p_K)$$

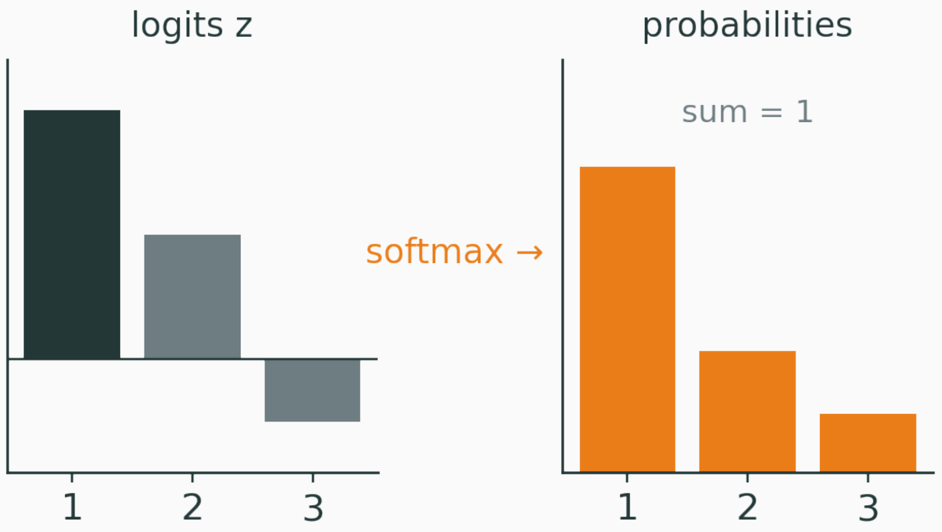
For a one-hot target y:

$$p(y \mid x, \theta) = \prod_{k=1}^{K} p_k^{y_k}$$

From logits to softmax

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Two guaranteed properties: $p_k > 0$ and $\sum_k p_k = 1$.



Shift invariance: $\mathrm{softmax}(z) = \mathrm{softmax}(z + c\mathbf{1})$.

Categorical NLL gives cross-entropy

D

$$-\log p(y \mid x, \theta) = -\log \prod_k p_k^{y_k} = -\sum_k y_k \log p_k$$

Because y is one-hot, only the true class survives:

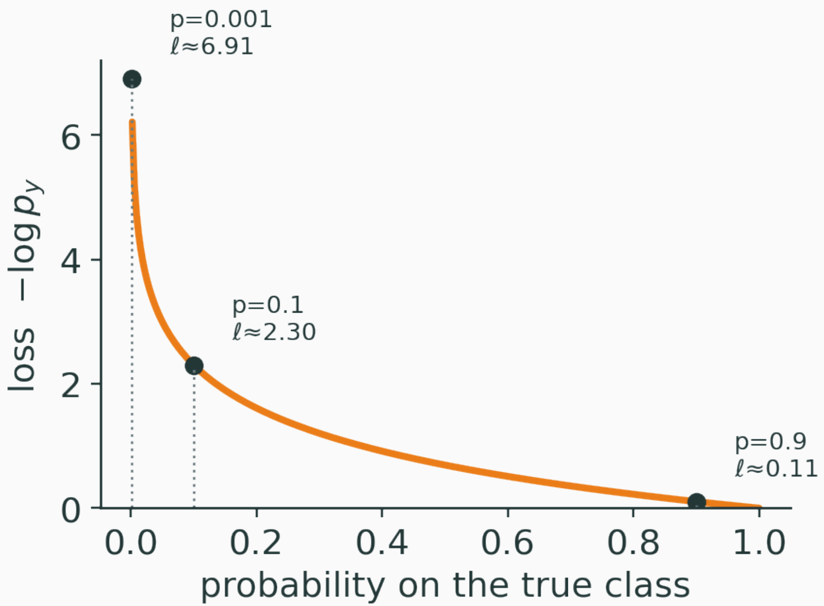
$$\mathcal{L} = -\log p_{\text{true class}}$$

Confident mistakes are expensive

V

$$\ell(p_y) = -\log p_y$$

$$p_y = 0.9 \Rightarrow \ell \approx 0.105$$
$$p_y = 0.1 \Rightarrow \ell \approx 2.303$$
$$p_y = 0.001 \Rightarrow \ell \approx 6.908$$

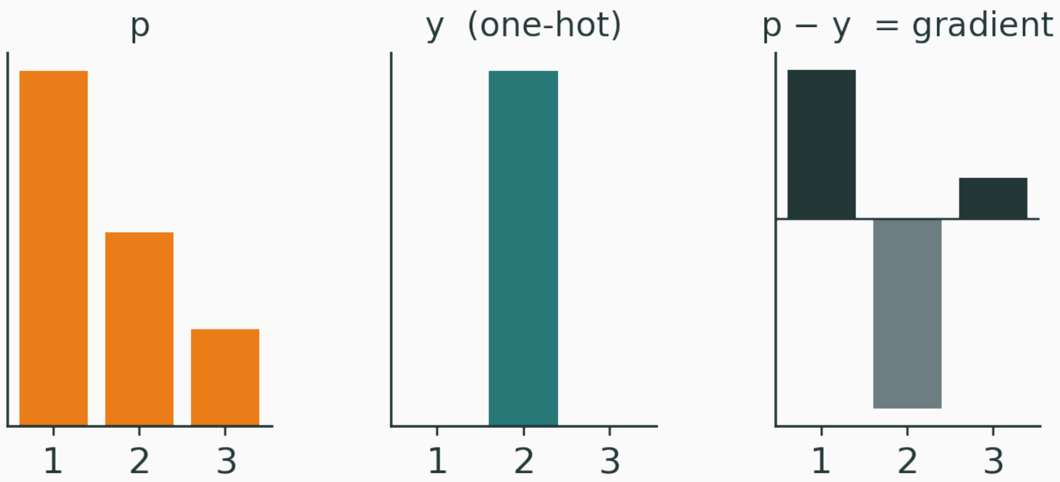


Q. Why punish a *confidently wrong* prediction so hard?

D

For softmax followed by cross-entropy:

$$\frac{\partial \mathcal{L}}{\partial z_k} = p_k - y_k, \qquad \nabla_z \mathcal{L} = p - y$$



Predicted p, target y, and their difference — a preview of backprop.

Interactive: logits → softmax → cross-entropy

I

INTERACTIVE

`softmax-cross-entropy.html` — controls z_1, z_2, z_3, the true class, and temperature T; presets for **correct & confident / correct & unsure / wrong & unsure / wrong & confident**. Shows $z \to \mathrm{softmax}(z/T) \to -\log p_y \to p - y$.

Ask students to predict the loss before it is revealed.

Why not MSE for classification?

optional

MSE is not *impossible* — it corresponds to the **wrong observation model** for class indicators.

MSE assumes $Y_k = p_k + \varepsilon_k, \varepsilon_k \sim \mathcal{N}(0, \sigma^2)$

Reality is $Y \sim \mathrm{Categorical}(p)$

Cross-entropy also gives **stronger gradients** on confident errors (where MSE's gradient vanishes).

Bayes’ rule, directly in classification

Bayes' rule as a classifier

$$p(y\{=\}k \mid x) = \frac{p(x \mid y\{=\}k)\, p(y\{=\}k)}{p(x)}$$

Prediction ignores the constant denominator:

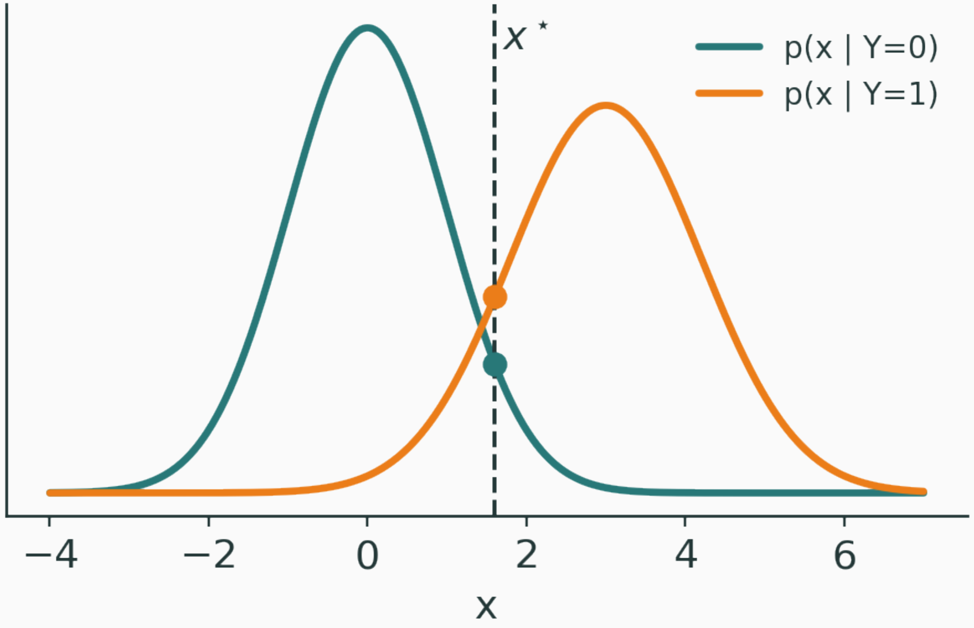
$$\hat{y} = \arg\max_k p(x \mid y\{=\}k)\, p(y\{=\}k)$$

- $p(y\{=\}k)$ — class **prior**
- $p(x \mid y\{=\}k)$ — class-conditional **likelihood**
- $p(y\{=\}k \mid x)$ — **posterior** class probability

A 1-D generative classifier

V

$$X \mid Y\{=\}0 \sim \mathcal{N}(\mu_0, \sigma_0^2), \quad X \mid Y\{=\}1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$$



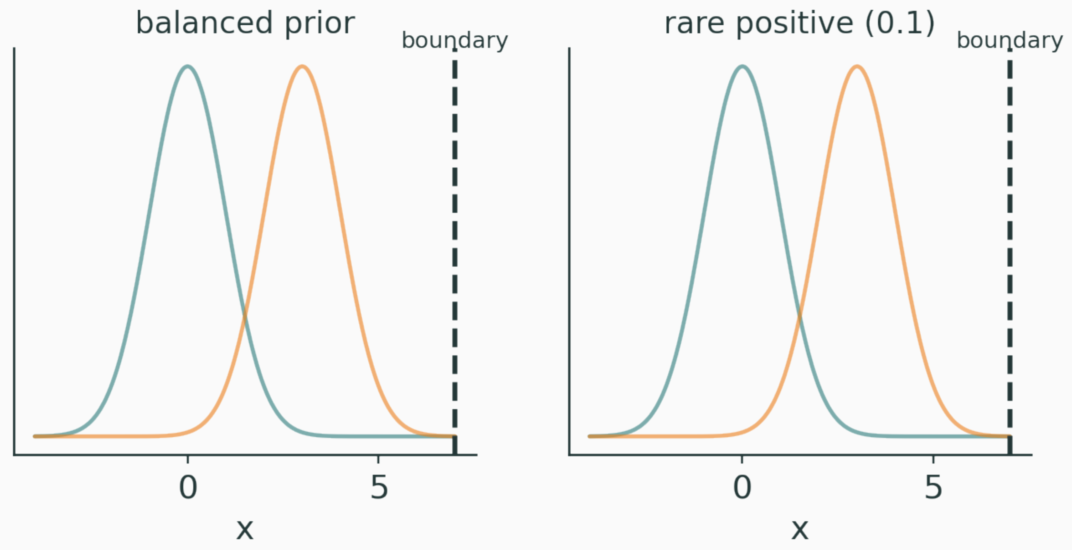
Two class-conditional densities, a test point $x^\star$, and its likelihood under each class.

Priors move the decision boundary

V

Balanced $p(Y\{=\}1) = 0.5$

Rare $p(Y\{=\}1) = 0.1$



posterior $\propto$ likelihood $\times$ prior

Interactive: Bayes classifier

I

INTERACTIVE

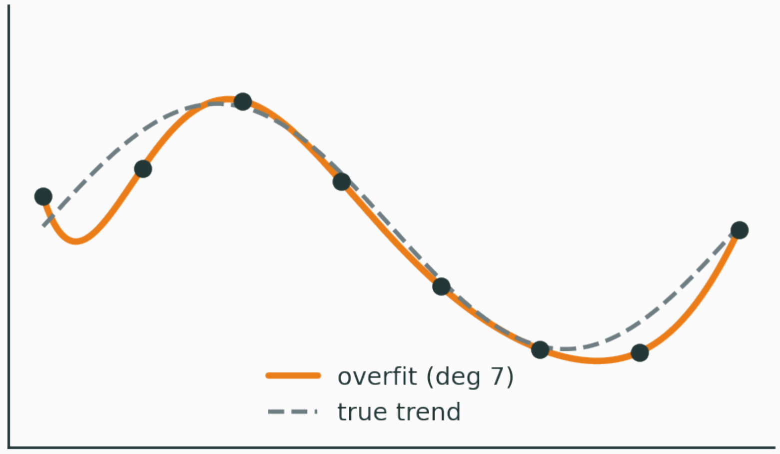
`bayes-classifier.html` — controls the means μ_0, μ_1, the variances, the **prior** $p(Y\{=\}1)$, and a test point $x^\star$. Shows $p(x \mid Y\{=\}0)$, $p(x \mid Y\{=\}1)$, the posterior $p(Y\{=\}1 \mid x)$, and the decision boundary together.

Try: balanced → drop the positive prior to 0.1 → why does the boundary move? then widen one variance.

Priors, MAP and regularization

The limitation of MLE

MLE asks only: *how well does θ explain the data?*



Many parameters fit a small dataset. **Which should we prefer?**

Put a prior over parameters

$$\theta \sim p(\theta)$$

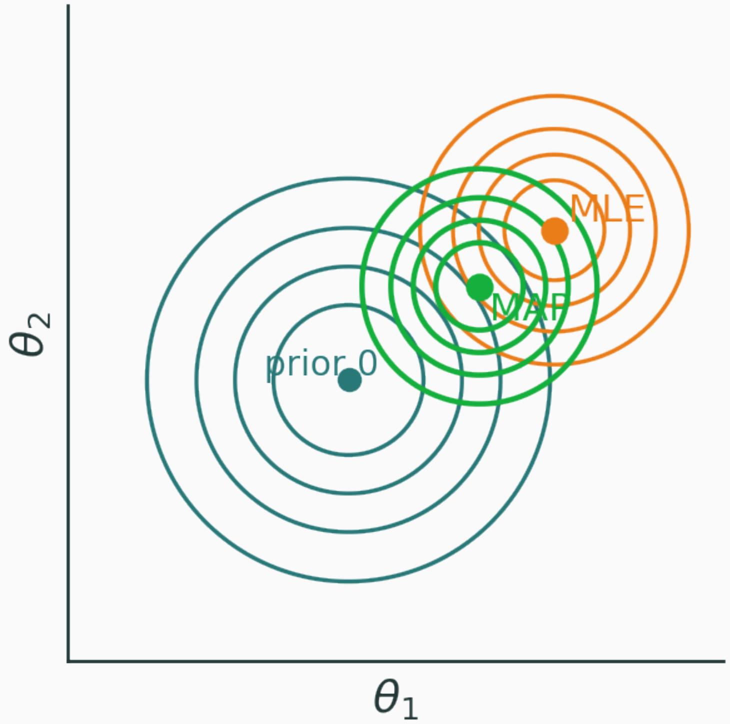
Before seeing this dataset, which parameter values are plausible?

- small weights · sparse weights · smooth functions · correlated parameters

A prior is an **inductive bias**.

Bayes' rule over parameters

$$p(\theta \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \theta)\, p(\theta)}{p(\mathcal{D})}$$



$p(\mathcal{D}) = \int p(\mathcal{D} \mid \theta) p(\theta)\, \mathrm{d}\theta$ — the evidence (we stop here).

MAP estimation

D

$$\hat{\theta}_{\text{MAP}} = \arg\max_{\theta} p(\theta \mid \mathcal{D}) = \arg\max_{\theta} p(\mathcal{D} \mid \theta)\, p(\theta)$$

Take negative logs:

$$\hat{\theta}_{\text{MAP}} = \arg\min_{\theta} (-\log p(\mathcal{D} \mid \theta) - \log p(\theta))$$

$$\boxed{\textbf{data loss} + \textbf{regularization}}$$

Gaussian prior produces L_2

D

Assume $\theta \sim \mathcal{N}(0, \tau^2 I)$. Then

$$-\log p(\theta) = \frac{1}{2\tau^2}\theta^\top \theta + C$$

$$\Rightarrow \quad \mathcal{L}_{\text{MAP}} = -\log p(\mathcal{D} \mid \theta) + \lambda\|\theta\|_2^2, \quad \lambda \propto \frac{1}{\tau^2}$$

narrow prior $\Longrightarrow$ **stronger** regularization; **broad** prior $\Longrightarrow$ **weaker**.

A general Gaussian prior → structured penalty

For $\theta \sim \mathcal{N}(0, \Sigma)$ the penalty is $\frac{1}{2}\theta^\top \Sigma^{-1} \theta$.

- isotropic Σ → ordinary L_2
- unequal variances → penalize parameters **differently**
- correlations → penalize particular **directions**

optional

$$p(\theta_j) = \frac{1}{2b}\exp\left(-|\theta_j\frac{|}{b}\right) \Rightarrow -\log p(\theta) = \frac{1}{b}\sum_j |\theta_j| + C$$

$\boxed{\textbf{Laplace prior} \Rightarrow L_1}$

Heavier tails at zero → **sparsity** (the lasso geometry, if time permits).

Interactive: likelihood, prior, MAP

I

INTERACTIVE

`map-regularization.html` — a two-parameter linear model so contours are visible. Shows **likelihood**, **prior**, and **posterior** contours with the **MLE**, **MAP**, and zero points.
Controls: amount of data · noise · prior variance · Gaussian vs Laplace · prior correlation.

Ask: more data? narrower prior? why does MAP → MLE as data grows? a correlated prior?

Full Bayes and the deep-learning objective

MAP is still one parameter vector

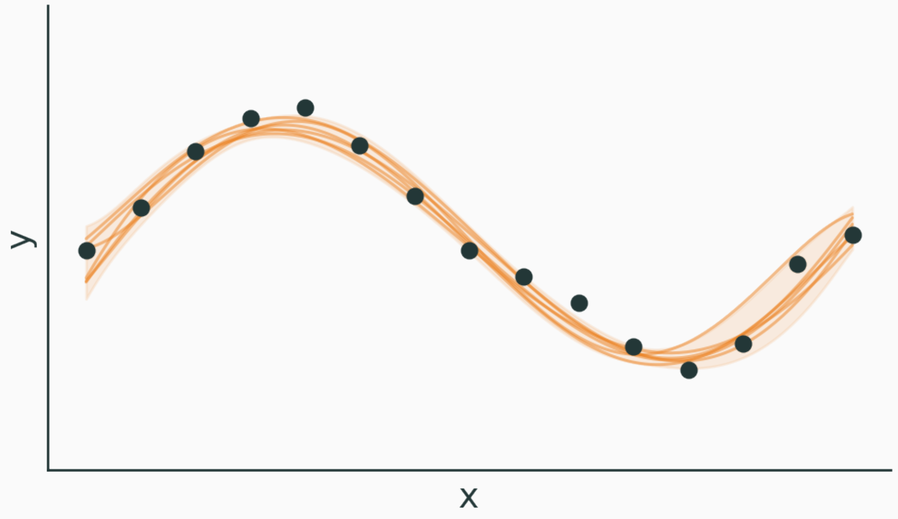
MLE $\hat{\theta}_{\mathrm{MLE}}$ — one point

MAP $\hat{\theta}_{\mathrm{MAP}}$ — one point

Full Bayes keeps the whole posterior $p(\theta \mid \mathcal{D})$ — a **distribution**, not a point.

Posterior predictive distribution

$$p(y^\star \mid x^\star, \mathcal{D}) = \int p(y^\star \mid x^\star, \theta)\, p(\theta \mid \mathcal{D})\, \mathrm{d}\theta$$



Bayesian prediction **averages over plausible models** — and reports uncertainty.

Why DL usually uses point estimates

optional

A modern network has millions–billions of parameters, so integrating $p(\theta \mid \mathcal{D})$ is hard. Common practical choices:

- MLE · MAP
- deep **ensembles**
- **dropout**-based approximations
- **variational** inference

The standard supervised DL objective

$$\min_{\theta} \underbrace{\frac{1}{|B|}\sum_{i\in B} -\log p_{\theta}(y_i \mid x_i)}_{\text{likelihood / data fit}} + \underbrace{\lambda\|\theta\|_2^2}_{\text{prior / regularization}}$$

probabilistic model + neural network + gradient optimization

DL doesn't discard probabilistic ML — it **scales** it with expressive functions and gradient descent.

Retrieval summary

Q

Complete each:

Gaussian likelihood $\Longrightarrow$? $\Longrightarrow$ **MSE**

Categorical likelihood $\Longrightarrow$? $\Longrightarrow$ **cross-entropy**

Gaussian prior $\Longrightarrow$? $\Longrightarrow$ $\boldsymbol{L_2}$

Laplace prior $\Longrightarrow$? $\Longrightarrow$ $\boldsymbol{L_1}$

MLE + prior $\Longrightarrow$? $\Longrightarrow$ **MAP**

Final mental model

See a **loss** → ask *“what likelihood produced it?”*
See a **regularizer** → ask *“what prior produced it?”*
See a **prediction** → ask *“point estimate or posterior?”*

Next: computation graphs, gradients, and backpropagation.